Predicción del precio de alojamientos de Airbnb en Estados Unidos

Trabajo final grupal. Ciencia de Datos con Python (I5521), UTN FRBA, 2026

Integrantes: Mora Romero, Fernanda Novoa De Melo, Ludmila Di Iacovo, Sofía Lara Mársico. Grupo: 1

1. Introducción y objetivos

Airbnb necesita estimar el precio por noche de publicaciones de distintas ciudades de Estados Unidos a partir de las características de la propiedad y del anfitrión. Se trata de un problema de regresión supervisada. Los objetivos del trabajo son:

- Explorar el dataset, detectar problemas de calidad de datos e identificar las variables más relacionadas con el precio.
- Construir un pipeline de Machine Learning reproducible que prediga el precio con cinco modelos: regresión lineal, Ridge, Lasso, Random Forest y XGBoost.
- Aplicar PCA como método de reducción de dimensionalidad y evaluar su efecto sobre la capacidad predictiva.
- Comparar los modelos con MAE, RMSE y R^2 sobre un conjunto de test que no participa del entrenamiento.

2. Descripción del dataset

- 19.309 publicaciones y 29 variables. La variable objetivo es *price*, el precio por noche en USD.
- Tipos de variables: categóricas (*property_type, room_type, bed_type, cancellation_policy, city*, entre otras), numéricas (*accommodates, bathrooms, bedrooms, beds, number_of_reviews, review_scores_rating, latitude, longitude*), texto (*name, description, amenities*), fechas (*host_since, first_review, last_review*) y un enlace (*thumbnail_url*).
- Valores faltantes: *host_response_rate* 22,25%, *review_scores_rating* 21,41%, *first_review* y *last_review* 20,48%, *thumbnail_url* 12,44%, *neighbourhood* 7,55% y *zipcode* 1,17%. Las variables *bathrooms, beds* y *bedrooms* tienen menos de 0,2%.
- Problemas de formato: las fechas y las columnas lógicas se importan como texto, *host_response_rate* incluye el símbolo "%" y existen valores 0 en *bathrooms* y *beds*.

3. Análisis exploratorio de datos

- **Precio:** la distribución es asimétrica hacia la derecha. La mayoría de las publicaciones está por debajo de USD 250 y hay una cola larga de valores extremos hasta USD 1999 (Figura 1). Al aplicar el logaritmo la distribución resulta mucho más simétrica.

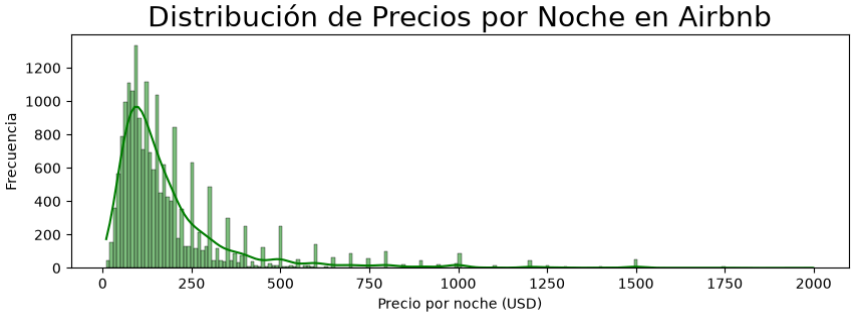


Figura 1. Distribución del precio por noche.

- **Tipo de habitación:** *Entire home/apt* tiene una mediana cercana a USD 180 y una dispersión mucho mayor que *Private room* y *Shared room* (Figura 2).

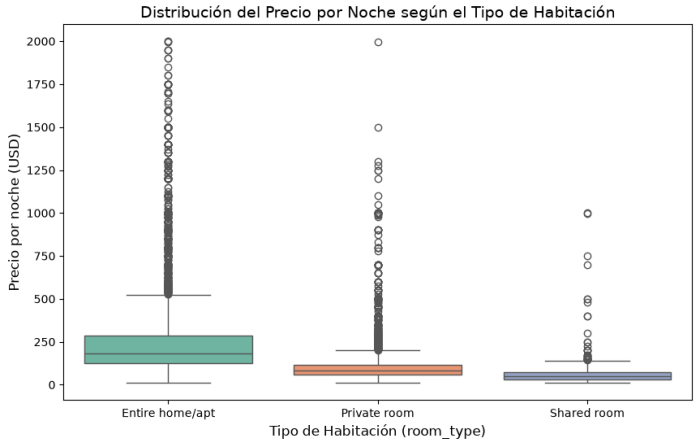


Figura 2. Precio por noche según el tipo de habitación.

- **Ciudad:** el dataset reúne cuatro ciudades. La mediana del precio por noche es de USD 99 en Chicago, USD 125 en DC, USD 136 en Boston y USD 165 en SF. SF y DC muestran más dispersión y valores extremos de hasta USD 2000, mientras que Chicago y Boston son más homogéneas.
- **Correlaciones con el precio:** *accommodates* 0,44, *bedrooms* 0,44, *beds* 0,37 y *bathrooms* 0,37. Latitud (−0,17) y longitud (−0,09) tienen valores bajos porque el dataset mezcla ciudades distintas. *id* (0,01) no aporta información.
- **Colinealidad:** las variables de tamaño están muy correlacionadas entre sí: *beds* y *accommodates* 0,83, *bedrooms* y *beds* 0,75, *bedrooms* y *accommodates* 0,74. Esto motiva el uso de regularización y de PCA.

4. Materiales y métodos

4.1 Preprocesamiento

- **Limpieza:** se eliminaron duplicados por *id* y las publicaciones con precio fuera de los percentiles 1 y 99 (se conserva el rango de USD 29 a USD 1100). Entre ambos criterios se eliminaron 360 filas y quedaron 18.949. Los valores 0 en *bathrooms* y *beds* se trataron como faltantes.
- **Variables nuevas:** *num_amenities* (cantidad de amenities de la publicación) y *antiguedad_host_dias* (días entre *host_since* y la fecha más reciente del dataset). *host_response_rate* se convirtió a número.
- **Variables descartadas:** *id, name, thumbnail_url, description* (identificadores o texto libre), *first_review, last_review* (alrededor de 20% de faltantes) y *neighbourhood, zipcode* (muchas categorías; la ubicación queda representada por *city, latitude* y *longitude*).
- **Pipeline:** en las variables numéricas se imputa con la mediana y se estandariza, $z = (x - \mu) / \sigma$. En las categóricas se imputa con la moda y se aplica one-hot. Las 20 variables predictoras (11 numéricas y 9 categóricas) generan 62 columnas. Todo se ajusta solo con los datos de entrenamiento para evitar fuga de información (data leakage).

- **Partición:** 80% entrenamiento (15.159 publicaciones) y 20% test (3.790), con semilla fija (random_state = 42).
- **Variable objetivo:** los modelos predicen log(precio) para reducir la asimetría. Las predicciones se llevan a USD con la exponencial y todas las métricas se calculan en USD.

4.2 Modelos

- **Referencia:** predice siempre la mediana del precio de entrenamiento.
- **Regresión lineal (mínimos cuadrados):** $\beta = (X^T X)^{-1} X^T y$. Es el modelo base.
- **Ridge:** minimiza $\Sigma(y_i - \hat{y}_i)^2 + \lambda \Sigma \beta_j^2$ (Hoerl y Kennard, 1970). La penalización L2 reduce los coeficientes.
- **Lasso:** minimiza $\Sigma(y_i - \hat{y}_i)^2 + \lambda \Sigma |\beta_j|$ (Tibshirani, 1996). La penalización L1 puede llevar coeficientes exactamente a cero. El valor de λ de Ridge y de Lasso se eligió por validación cruzada de 5 folds.
- **Random Forest:** 100 árboles de decisión entrenados sobre muestras bootstrap cuyas predicciones se promedian (Breiman, 2001).
- **XGBoost:** boosting de árboles, donde cada árbol corrige los errores de los anteriores (Chen y Guestrin, 2016). Los hiperparámetros se buscaron con RandomizedSearchCV (50 combinaciones, validación cruzada de 3 folds, error cuadrático medio).

4.3 Reducción de dimensionalidad

PCA proyecta los datos estandarizados sobre los autovectores de la matriz de covarianza, ordenados por autovalor. Se retienen las primeras componentes hasta explicar el 95% de la varianza. PCA forma parte del pipeline (se ajusta solo con entrenamiento) y se vuelven a entrenar los cinco modelos sobre las componentes.

4.4 Métricas

- $MAE = (1/n) \Sigma |y_i - \hat{y}_i|$
- $RMSE = \sqrt{((1/n) \Sigma (y_i - \hat{y}_i)^2)}$
- $R^2 = 1 - \Sigma (y_i - \hat{y}_i)^2 / \Sigma (y_i - \bar{y})^2$

5. Experimentos y resultados

Hiperparámetros elegidos: λ de Ridge 0,01; λ de Lasso 0,00036; XGBoost con 500 árboles, profundidad 5, learning rate 0,05, subsample 0,8 y colsample_bytree 0,7. Con PCA se pasó de 62 columnas a 17 componentes, que explican el 95,8% de la varianza (Figura 3).

Modelo	MAE (USD)	RMSE (USD)	R^2
Referencia (mediana)	103,64	177,81	−0,131
Regresión lineal	72,72	135,63	0,342
Ridge	72,78	135,65	0,342
Lasso	73,07	135,99	0,338
Random Forest	65,00	123,12	0,458
XGBoost	62,81	118,74	0.50

Modelo	MAE (USD)	RMSE (USD)	R^2
Regresión lineal + PCA	74,89	136,96	0,329
Ridge + PCA	74,90	136,98	0,329
Lasso + PCA	74,91	137,06	0,328
Random Forest + PCA	70,72	130,80	0,388
XGBoost + PCA	69,69	127,74	0,416

Tabla 1. Métricas sobre el conjunto de test (3.790 publicaciones), en USD.

- XGBoost obtuvo el mejor desempeño: reduce el MAE un 39% y el RMSE un 33% respecto de la referencia. Random Forest quedó segundo.
- Regresión lineal, Ridge y Lasso rindieron prácticamente igual (R^2 cercano a 0,34).
- Lasso llevó a cero 25 de las 62 columnas: tipos de propiedad poco frecuentes, *latitude*, *instant_bookable* y *host_identity_verified*, entre otras.
- Según XGBoost, las variables más importantes son el tipo de habitación (*Entire home/apt* con importancia cercana a 0,45 y *Private room* cercana a 0,11), seguidas por *bedrooms*, *city_SF* y *bathrooms*.
- Efecto de PCA sobre el RMSE: +1,3 USD en los tres modelos lineales, +7,7 USD en Random Forest y +9,1 USD en XGBoost.
- El gráfico de precios reales contra predichos muestra que el modelo subestima los precios altos (Figura 4).

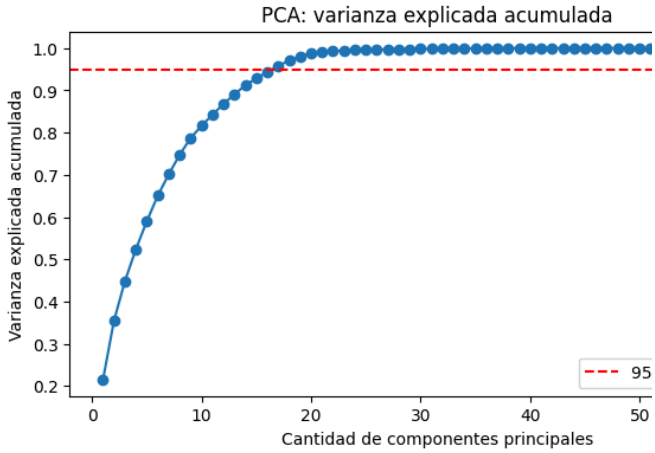


Figura 3. Varianza explicada acumulada por PCA.

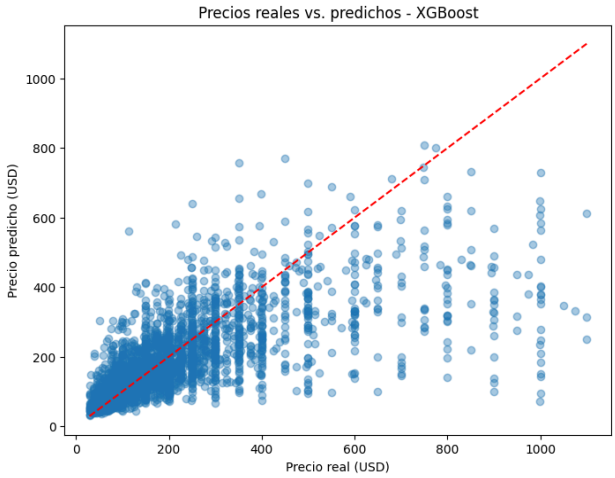


Figura 4. Precios reales contra predichos (XGBoost).

6. Discusión y conclusiones

- **Árboles contra modelos lineales:** Random Forest y XGBoost superaron claramente a los modelos lineales. Una explicación posible es que el precio depende de combinaciones entre variables (por ejemplo ciudad y tipo de habitación) que un modelo lineal no captura. No se contrastó esta hipótesis de forma explícita.
- **Regularización:** Ridge y Lasso no mejoraron a la regresión lineal. Con 20 variables y más de 15.000 observaciones hay poco sobreajuste que corregir, y Ridge eligió el λ más bajo de la grilla. Lasso fue útil para identificar variables que aportan poco.

- **PCA:** redujo las dimensiones un 73% (de 62 a 17) con una pérdida de cerca de 1% en el RMSE de los modelos lineales, pero empeoró entre 6% y 8% el RMSE de los modelos de árboles. PCA es no supervisado, resume la varianza de las variables sin considerar el precio y rota los ejes originales, lo que afecta a los árboles porque estos cortan sobre variables originales como el tipo de habitación o la ciudad. Por eso PCA no mejoró los resultados; esto no indica que esté mal aplicado, sino que con esta configuración conservar las variables originales predice mejor.
- **Limitaciones:** el R^2 máximo es 0,50, por lo que cerca de la mitad de la variabilidad del precio queda sin explicar, y el modelo subestima los precios altos. El filtrado de outliers se hizo antes de la partición, por lo que el test también excluye precios extremos y las métricas no son comparables con las de un modelo evaluado sobre todos los precios. La búsqueda de hiperparámetros fue acotada (el número de árboles elegido es el máximo de la grilla) y se descartaron variables como *neighbourhood, zipcode* y *description*, que podrían aportar información.
- **Trabajo futuro:** incorporar el barrio con una codificación adecuada, extraer variables del texto de la descripción, ampliar la búsqueda de hiperparámetros y probar otros modelos de boosting.
- **Conclusión:** XGBoost sin PCA fue el modelo recomendado (MAE de USD 62,83 y R^2 de 0,497 en test). Todos los modelos superaron a la referencia de la mediana.

7. Referencias

Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.

Chen, T. y Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.

Hoerl, A. E. y Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1), 55-67.

James, G., Witten, D., Hastie, T. y Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2.ª ed.). Springer.

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Tibshirani, R. (1996). Regression Shrinkage and Selection via the Lasso. *Journal of the Royal Statistical Society: Series B*, 58(1), 267-288.